ASSIGNMENT 5 – PART B – TASK B2

ENGINEER AN ADVERSARIAL INPUT

CO5 – Data Persistence Engineering | 5 Marks | Bloom's Level K4 – Analyse

Objective. Demonstrate that a crafted login input can bypass an unsafe concatenated SQL query, show that PreparedStatement rejects the same invalid credentials, and test the response of a five-field bus-pass file reader to damaged records.

1. SQL injection test

The local assignment-only database contains one valid account: student01 / pass123. The test supplies the ID `' OR '1'='1' --` and the incorrect password `definitely-wrong`.

The unsafe implementation concatenates values into SQL. The resulting statement is equivalent to:

```
SELECT * FROM users WHERE id='' OR '1'='1' -- ' AND pwd='definitely-wrong'
```

The OR expression is always true, and the comment marker removes the remainder of the condition. The unsafe query therefore returns a row. PreparedStatement uses WHERE id=? AND pwd=? and binds the attack text as a literal value, so it cannot modify SQL structure and the invalid login is rejected.

Test	Expected result	Reason
Unsafe + attack ID + wrong password	true (vulnerable)	Injected OR condition matches a row
PreparedStatement + same input	false (rejected)	Attack string remains parameter data
PreparedStatement + valid credentials	true	Stored test credentials match

2. Damaged bus-pass file

The record format has five fields: pass ID, register number, name, route number and amount. The supplied test file has two valid records, one row with four fields, one row with a non-numeric amount and one empty line.

Line	Damage / input	Handling
1	Valid five fields	Accepted
2	Four fields only	Skipped: wrong field count
3	Amount = three-eight	Skipped: invalid amount
4	Empty line	Skipped: empty
5	Valid five fields	Accepted

Expected summary: 2 valid rows accepted; 1 wrong-field-count row, 1 invalid-amount row and 1 empty line skipped. The reader continues after damaged rows, so a malformed record does not prevent later valid records from being processed.

3. What each input reveals

SQL injection reveals that concatenating untrusted input mixes executable SQL with user data. Wrong field count reveals that the row cannot be mapped to the expected schema. A non-numeric amount reveals missing type validation and possible parsing or financial-data errors. An empty line must be distinguished from a record. Continuing the scan demonstrates fault isolation.

4. Alternative: escaping or regex validation

Quote escaping is database- and context-dependent; incomplete escaping can reintroduce injection. Regex validation is brittle as accepted formats evolve and cannot safely cover every SQL context. PreparedStatement is stronger because the SQL structure is fixed and values are bound separately.

5. Complexity analysis

Let m be input length and n the number of user rows. Manual escaping or regex validation takes O(m) per query, in addition to the database lookup. Parameter binding also processes parameter data in O(m), but avoids manual SQL rewriting. Database lookup depends on the query plan: typically O(log n) with a suitable index or O(n) for a full scan. PreparedStatement improves security and correctness; it does not change asymptotic lookup complexity.

6. Execution procedure

Use Java 11 or later. Download H2 2.3.232 and place h2-2.3.232.jar in lib/. Compile with javac -d output src\B2AdversarialTest.java and run with java -cp "output;lib\h2-2.3.232.jar" B2AdversarialTest. The standalone damaged-file test can be run without H2 using javac -d output src\B2DamagedFileTest.java followed by java -cp output B2DamagedFileTest.

Evidence note: The expected transcript is a reference, not proof of execution. Run the program on the target computer and capture the actual terminal output. This report intentionally excludes student-identifying details.